



Republic of the Philippines
CAVITE STATE UNIVERSITY
Bacoor City Campus
SHIV, Molino VI, City of Bacoor

IoT Sensors and Actuators

EDAN A. BELGICA
INSTRUCTOR I

Table of Contents

COURSE DESCRIPTION..... 2

MISSION 2

VISION 2

COURSE REQUIREMENTS 2

INTENDED LEARNING OUTCOMES 3

PRE-TEST: 3

Digital Sensors and Analog Sensors 5

Analog Sensors..... 5

Digital Sensors 5

LDR — Light-Dependent Resistor..... 6

DHT11 — Temperature and Humidity Sensor 7

PIR Sensor — Motion Detection..... 9

OLED Display..... 11

Relay — Actuator..... 13

Servo Motor — Actuator 14

Buzzer — Actuatorbuzz 15

POST-TEST:..... 17

ANSWER KEY:..... 19

REFERENCES..... 20



Republic of the Philippines

CAVITE STATE UNIVERSITY

Bacoor City Campus

SHIV, Molino VI, City of Bacoor

MISSION

Cavite State University shall provide excellent, equitable and relevant educational opportunities in the arts, science and technology through quality instruction and relevant research and development activities. It shall produce professional, skilled and morally upright individuals for global competitiveness.

VISION

The premier university in historic Cavite recognized for excellence in the development of globally competitive and morally upright individuals.

Edan A. Belgica
Instructor I
edan.belgica@cvsu.edu.ph

COURSE DESCRIPTION

COSC 111: INTERNET OF THINGS

This course aims to help students to understand the IoT and its applications, apply the web of things and cloud of things, do IoT hardware and IoT sensors interfacing, and develop IoT applications of real world.

PROGRAM OUTCOMES ADDRESSED BY THE COURSE. AFTER COMPLETING THIS COURSE, THE STUDENTS MUST BE ABLE TO:

1. Attain the vision, mission, goals and objectives of the university, campus and department; (E)
2. Deliver a gender fair and gender sensitive instruction to students aligned with University goals and objectives. (D)
3. Explain the concepts, architecture, and applications of the Internet of Things.
4. Develop Arduino/ESP32-based embedded applications using sensors, actuators, and communication modules.
5. Integrate IoT devices with web services, cloud platforms, and data visualization tools for real-time monitoring and control.
6. Evaluate the security, privacy, ethical, and societal implications of Internet of Things applications.
7. Design, implement, test, document, and present a real-world IoT solution that addresses a practical problem using appropriate computing tools.

COURSE REQUIREMENTS

1. Homework/Activity
2. Long Examination
3. Midterm and Final Examination



INTENDED LEARNING OUTCOMES

After the completion of the unit, students will be able to:

1. Interface sensors with the ESP32.
2. Read and process sensor data.
3. Control actuators based on sensor input.

PRE-TEST:

Direction: Read each question carefully and select the letter corresponding to the correct answer.

1. Which function reads continuous voltage variations from an analog sensor on an ESP32?
 - A) digitalRead()
 - B) analogRead()
 - C) analogWrite()
 - D) digitalWrite()
2. Which communication protocol uses two lines—SDA (Data) and SCL (Clock)—to interface modules with the microcontroller?
 - A) SPI
 - B) Single-Wire
 - C) I2C
 - D) UART
3. How does a Light-Dependent Resistor (LDR) behave as light intensity increases?
 - A) Resistance increases
 - B) Resistance decreases
 - C) Voltage drops to zero continuously
 - D) Resistance stays completely fixed
4. Which hardware component allows a low-power microcontroller signal to switch high-power circuits?
 - A) Relay
 - B) Passive Infrared Sensor
 - C) Light-Dependent Resistor
 - D) Active Buzzer
5. What type of signal does a Passive Infrared (PIR) sensor output when motion is detected?
 - A) Continuously varying analog voltage
 - B) High-speed SPI data packets

- C) Digital HIGH logic level
 - D) Variable frequency PWM output
6. What is the typical logic operating voltage range for ESP32 GPIO pins?
- A) 0 V to 5 V
 - B) 0 V to 3.3 V
 - C) 0 V to 12 V
 - D) -5 V to +5 V
7. Which Arduino IDE function is used to set a pin as an input or output?
- A) pinMode()
 - B) digitalWrite()
 - C) analogRead()
 - D) Serial.begin()
8. Which device converts an electrical signal into an audible sound in an IoT system?
- A) LDR
 - B) Buzzer
 - C) OLED Display
 - D) PIR Sensor
9. What data protocol is used by the DHT11 sensor to transmit temperature and humidity data?
- A) I2C protocol
 - B) Single-Wire custom pulse protocol
 - C) SPI dual-line protocol
 - D) Analog voltage output
10. What software component is required in Arduino IDE to interface complex sensors that use specialized protocols?
- A) Bootloader
 - B) Library
 - C) Compiler
 - D) Baud Rate

DIGITAL SENSORS AND ANALOG SENSORS

An **Arduino** is an open hardware development board that can be used by tinkerers, hobbyists, and makers to design and build devices that interact with the real world. While Arduino refers to a specific type of board design, it can also be used to refer to a company which manufactures a specific implementation of these boards, and is typically also used to describe the community around compatible boards made by other people or companies which function in a similar way.

ANALOG SENSORS

Measuring Continuous Physical Reality

In nature, physical phenomena like temperature, light level, sound pressure, and force change smoothly and continuously across an infinite number of possible values. Analog sensors reflect this directly by changing an electrical property—most commonly **resistance** or **voltage**—in step with the environment.

They are read using `analogRead()`, which converts the voltage into a digital value via the ESP32's ADC (Analog-to-Digital Converter).

Electrical Behavior

An analog sensor continuously outputs a voltage between 0 V and its operating voltage (V_{CC} , typically 3.3 V or 5 V).

- **Light-Dependent Resistor (LDR):** Drops resistance as light increases. When wired in a voltage divider circuit, the output node changes voltage smoothly as room light changes.
- **Potentiometer:** Functions as a manually adjustable voltage divider outputting anywhere from 0 V to V_{CC} .

DIGITAL SENSORS

On-Off States & Serial Protocols

Digital sensors output data that is already digitized. They fall into two main categories: **simple binary state sensors** and **integrated protocol sensors**.

Category A: Simple Binary (State-Based) Sensors

These sensors output only two logic voltage levels: **LOW** (0 V) or **HIGH** (V_{CC} , e.g., 3.3 V).

- **PIR Motion Sensor:** Outputs 3.3 V when infrared motion is detected, and 0 V when clear.
- **Pushbutton / Limit Switch:** Connects or breaks a circuit path to toggle a pin state.

The microcontroller uses `digitalRead(pin)`, which evaluates voltage against fixed logic thresholds (e.g., < 0.8 V is LOW, > 2.0 V is HIGH).

Category B: Complex Digital / Protocol Sensors

Many modern sensors (e.g., DHT11, BME280, MPU6050) contain an internal sensing element **plus a built-in micro-controller/ADC chip** inside the sensor module itself.

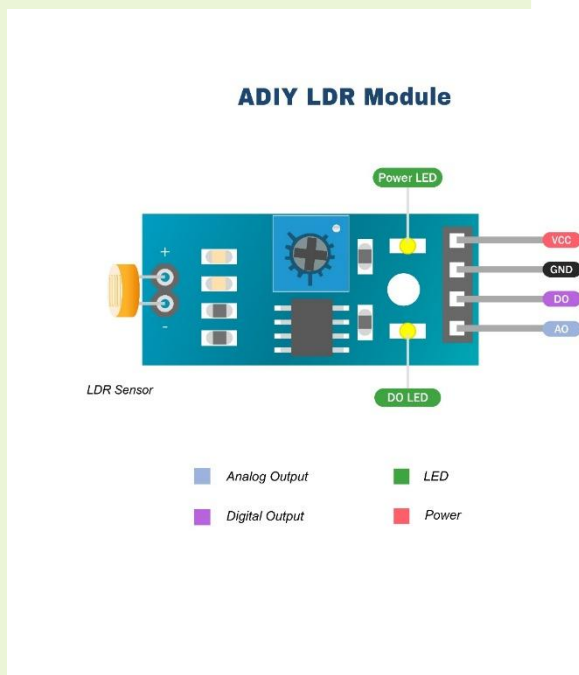
- The internal element measures the raw analog signal.

- The internal chip performs the analog-to-digital conversion and applies factory calibration.
- The sensor transmits the final readings (e.g., "24.5°C, 60% RH") over a digital data line using a standardized protocol:
 - a. **Single-Wire / Custom Protocol (DHT11/DHT22):** Sends timed high/low pulse trains down a single wire.
 - b. **I²C (Inter-Integrated Circuit - BME280, MPU6050):** Uses two lines (SDA data, SCL clock) to allow multiple sensors on the same bus.
 - c. **SPI / UART:** High-speed multi-wire communication protocols.

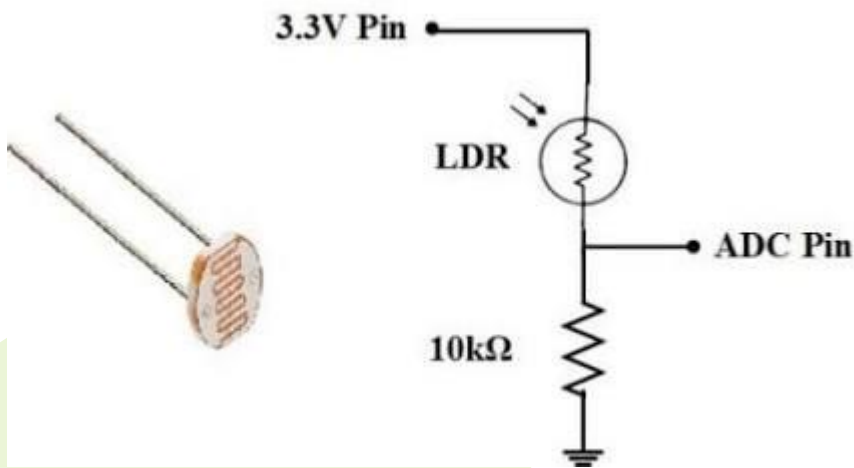
They are read using `digitalRead()` or, for more complex sensors, dedicated communication protocols and libraries.

LDR — LIGHT-DEPENDENT RESISTOR

An LDR is a resistor whose resistance decreases as light intensity increases (and increases in darkness). Paired with a fixed resistor in a voltage divider circuit, it produces a variable voltage that the ESP32 can read as an analog value.



An LDR (Light Dependent Resistor or photoresistor) utilizes photoconductivity to convert light intensity into an electrical value. Made from semiconductor material like cadmium sulfide, photons hitting the component release bound electrons, increasing conductivity and causing resistance to plummet.



Reading Light Levels

```
int ldrPin = 34;

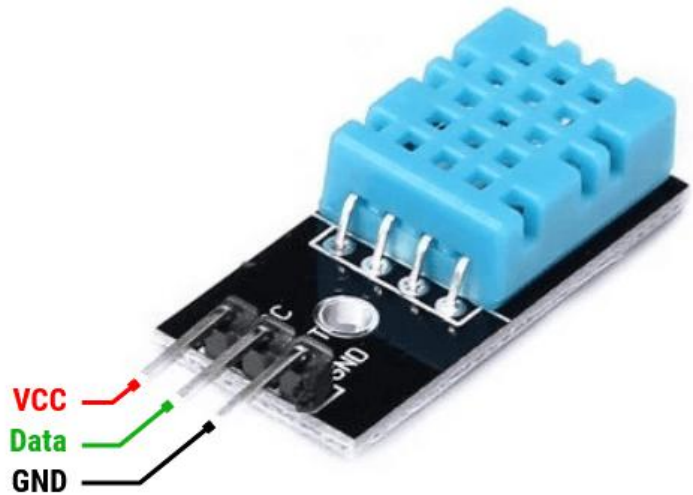
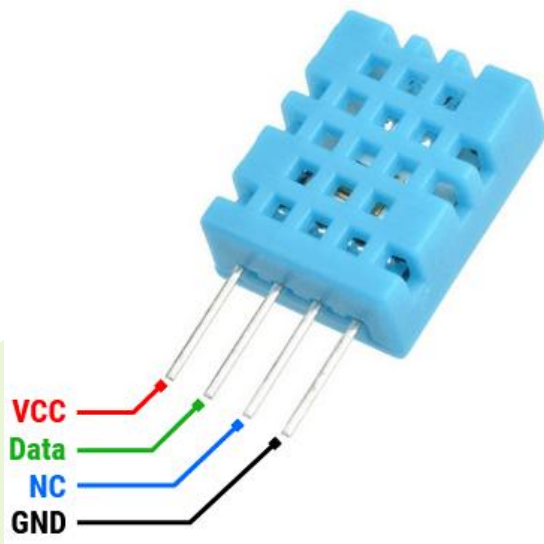
void setup() {
  Serial.begin(115200);
}

void loop() {
  int lightValue = analogRead(ldrPin);
  Serial.print("Light Level: ");
  Serial.println(lightValue);
  delay(300);
}
```

Observe and record how the reading changes when covering the LDR with a hand versus exposing it to a light source (e.g., phone flashlight).

DHT11 — TEMPERATURE AND HUMIDITY SENSOR

The DHT11 is a digital sensor that measures both temperature and relative humidity, communicating with the microcontroller over a single-wire digital protocol. It requires a dedicated library to decode its signal correctly.



The DHT11 combines a capacitive humidity sensor, a thermistor for temperature measurement, and an internal 8-bit chip that processes these analog readings into a digital signal.

Reading Temperature and Humidity

Search for "DHT sensor library" (by Adafruit) and install it, along with its dependency "Adafruit Unified Sensor" if prompted.

```
#include <DHT.h>

#define DHTPIN 15
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(115200);
  dht.begin();
}

void loop() {
  float temperature = dht.readTemperature();
  float humidity = dht.readHumidity();

  if (isnan(temperature) || isnan(humidity)) {
    Serial.println("Failed to read from DHT sensor!");
  } else {
    Serial.print("Temperature: ");
    Serial.print(temperature);
    Serial.print(" °C Humidity: ");
```

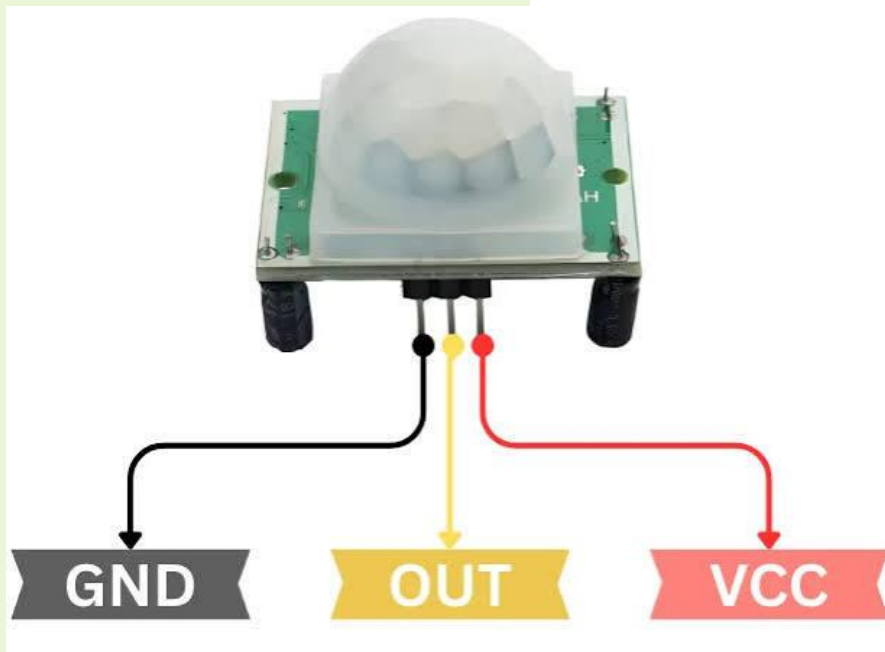
```

Serial.print(humidity);
Serial.println(" %");
}
delay(2000);
}

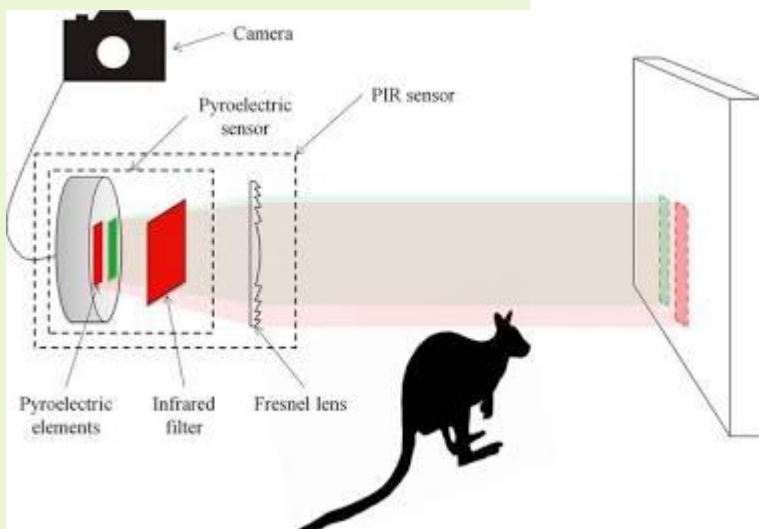
```

PIR SENSOR — MOTION DETECTION

A Passive Infrared (PIR) sensor detects motion by sensing changes in infrared radiation (heat) within its field of view. It outputs a simple digital HIGH signal when motion is detected and LOW otherwise.



A PIR sensor is classified as "passive" because it produces no energy or light beams of its own; it solely measures the infrared radiation emitted by objects (like humans or animals) in its environment.



Internal Components & Differential Detection

- **Pyroelectric Sensor:** Behind the plastic dome sits a metal can housing two side-by-side IR-sensitive slots made of crystalline material (such as gallium nitride or lithium tantalate).
- **Differential Voltage:** When the room is static, both slots receive equal amounts of ambient infrared radiation, producing zero net signal.
- **Movement Detection:** When a warm body walks past:
 1. The body intercepts the first slot, causing a positive differential voltage spike.
 2. The body moves across to the second slot, causing a negative differential voltage spike.
 3. The internal IC (like the BISS0001) processes this differential change and triggers the signal output pin high (3.3V).

Role of the White Plastic Dome (Fresnel Lens)

The translucent dome covering the sensor is a Fresnel lens. It serves two functions:

1. **Focusing Light:** It gathers infrared energy from a wide detection cone (typically $110^\circ - 120^\circ$) and focuses it onto the tiny internal pyroelectric chip.
2. **Splitting Fields:** It breaks the field of view into multiple distinct beam zones separated by blind spots. As a warm body moves through these interchanging zones, it amplifies the signal change felt by the sensor slots.

On-Board Calibration Controls (HC-SR501)

Common PIR breakout boards feature two potentiometers and a jumper setting:

- **Sensitivity Adjustment:** Controls detection distance range (typically adjustable between 3 meters and 7 meters).
- **Time Delay Adjustment:** Controls how long the output pin stays HIGH after detecting motion (from ~3 seconds up to 5 minutes).
- **Trigger Mode Jumper:**
 - **Non-Repeatable (L):** Output goes LOW immediately after the timer expires, even if motion continues.
 - **Repeatable (H):** Output stays HIGH continuously as long as motion is detected within the sensing area.

Motion-Triggered LED

```
int pirPin = 27;
```

```
int ledPin = 2;
```

```
void setup() {  
  pinMode(pirPin, INPUT);  
  pinMode(ledPin, OUTPUT);  
  Serial.begin(115200);  
}
```

```

}

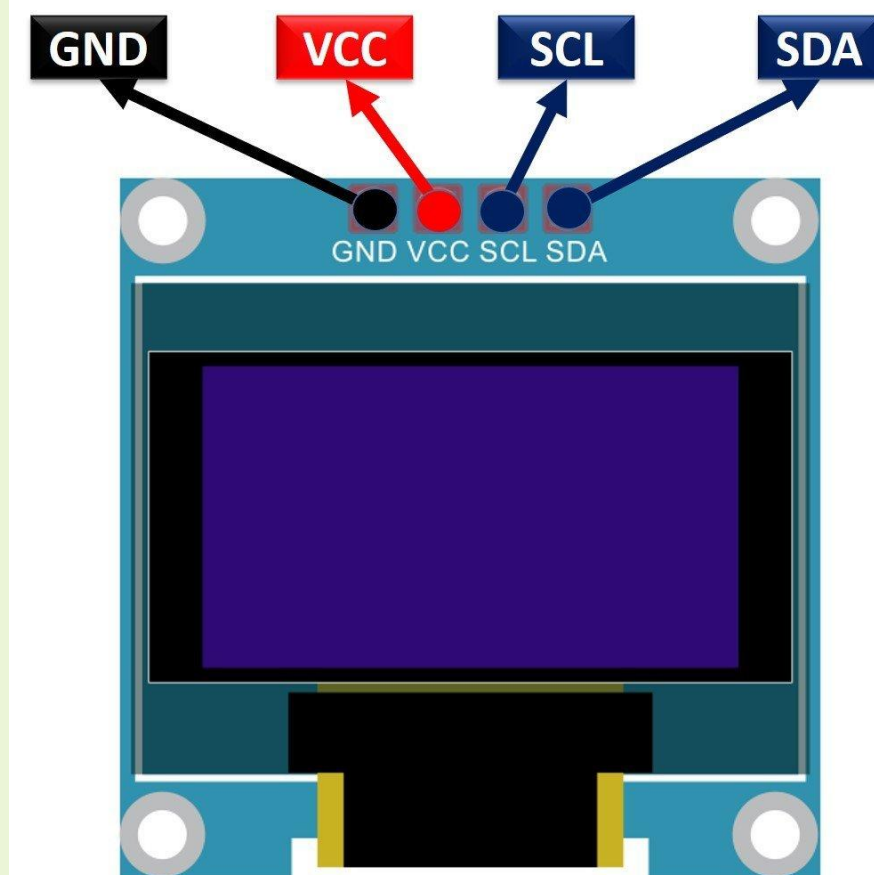
void loop() {
  int motionState = digitalRead(pirPin);
  if (motionState == HIGH) {
    digitalWrite(ledPin, HIGH);
    Serial.println("Motion Detected!");
  } else {
    digitalWrite(ledPin, LOW);
  }
  delay(200);
}

```

Test the sensor's detection range and sensitivity by moving at different distances

OLED DISPLAY

An OLED (Organic Light-Emitting Diode) display is commonly used in IoT projects to show sensor readings or system status directly on the device, without needing a Serial Monitor. Common modules communicate via the I2C protocol (typically using SDA(serial data) and SCL(serial clock) pins).



Unlike traditional LCD displays that require a dedicated LED backlight array, an OLED display relies on individual self-emissive organic pixel nodes. When current passes through a pixel, it illuminates independently, delivering true black levels, exceptional contrast ratios, and significantly reduced overall power draw.

I2C Bus Architecture & Addressing

The Inter-Integrated Circuit (I^2C) bus allows microcontrollers like the ESP32 to communicate with the OLED display using only two signal lines:

- SDA (Serial Data): Handles bidirectional transmission of screen buffer data, commands, and configuration registers.
- SCL (Serial Clock): Synchronizes timing pulses sent from the master device (ESP32) to the peripheral display.
- Device Addressing: Most 0.96-inch SSD1306-driven OLED panels operate on hex address 0x3C (or occasionally 0x3D). Because I^2C uses addressing, multiple devices (e.g., an OLED, RTC, and pressure sensor) can share the exact same SDA and SCL physical pins.

Installing Required Libraries

- Install **Adafruit SSD1306** and **Adafruit GFX Library** via the Library Manager.

Displaying Sensor Data on OLED

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

void setup() {
  Serial.begin(115200);
  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("OLED not found");
    while (true);
  }
  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
}

void loop() {
```

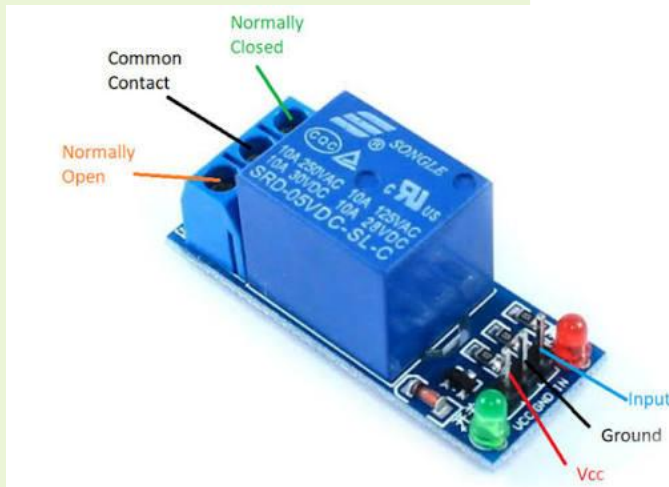
```

display.clearDisplay();
display.setCursor(0, 0);
display.println("Hello, IoT!");
display.display();
delay(1000);
}

```

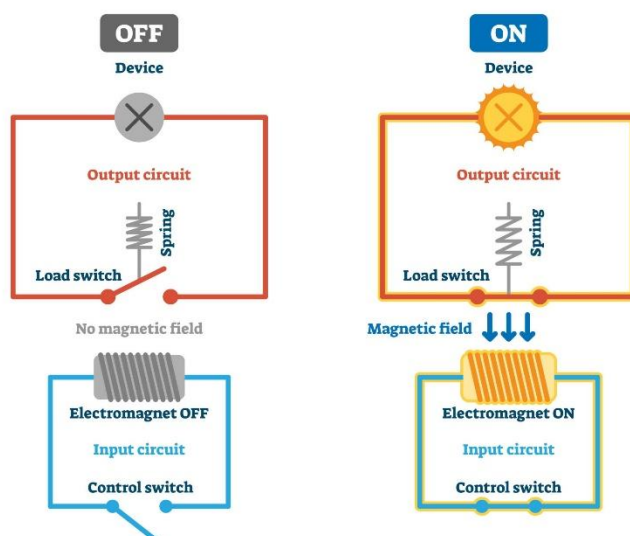
RELAY — ACTUATOR

A relay is an electrically operated switch that allows a low-power microcontroller signal (from the ESP32) to control a much higher-power circuit (e.g., a lamp, fan, or appliance) by physically opening or closing a set of contacts.



A relay acts as a bridge between two completely separated electrical systems: a low-power control side (like an ESP32 output pin operating at 3.3V) and a high-power load side (like a 220V AC wall outlet powering a lamp).

SIMPLE RELAY DIAGRAM



How It Works Internally

1. Low-Power Control Signal: When a GPIO pin sends a HIGH signal, a small current flows through a wire coil inside the relay.
2. Electromagnetic Activation: This current transforms the coil into an electromagnet, generating a magnetic field.
3. Physical Switching: The magnetic force mechanically pulls a movable metal arm (armature), snapping the internal switch contacts closed or open.
4. Complete Galvanic Isolation: Because the physical contact touches the high-voltage side while the coil handles the low-voltage side, no high voltage can flow backward to damage the microcontroller.

Relay Module Terminal Pins

Standard relay breakout modules feature three main screw terminals on the high-voltage load side:

- COM (Common): Connected to one leg of the power supply for the device you want to control.
- NO (Normally Open): The circuit stays disconnected by default. Sending a HIGH signal closes the circuit and turns the device ON.
- NC (Normally Closed): The circuit stays connected by default. Sending a HIGH signal opens the circuit and turns the device OFF.

SERVO MOTOR — ACTUATOR

A servo motor is an actuator that rotates to a specific, precisely controllable angle (commonly 0°–180° for hobby servos), rather than simply spinning continuously like a DC motor. It is controlled by sending it a repeating PWM pulse: the *width* of the pulse tells the servo which angle to hold. Servos are useful in IoT projects wherever a sensor reading should translate into a *physical, positional* response — for example, opening a vent, rotating a small barrier, adjusting a solar panel's angle, or pointing a camera/sensor mount.

Power Note: A single small SG90-class servo can typically run directly off the ESP32's 5V pin for light, brief movements. However, if the servo stalls, moves under load, or the board resets/browns-out when the servo moves, power it from a separate 5V source and connect the grounds of the ESP32 and the external supply together (common ground).

Pin Note: Avoid GPIO pins in the ADC2 group for the servo's signal pin, since ADC2 pins can be unreliable while Wi-Fi is active on the ESP32.

Installing the Library

Search for "ESP32Servo" and install it. This library is written specifically to work correctly with the ESP32's PWM/LEDC hardware (the standard Arduino Servo.h library is not ESP32-compatible).

BUZZER — ACTUATORBUZZ

A buzzer is an actuator that converts an electrical signal into an audible sound, commonly used for alerts and notifications in IoT systems (e.g., alarms, reminders).



Buzzers convert electrical signals into mechanical sound waves using either piezoelectric or electromagnetic mechanisms. In microcontroller applications, choosing between an active or passive buzzer determines whether sound generation is handled by internal hardware or micro-controlled signal timing.

Active vs. Passive Buzzers

- Active Buzzer: Contains an internal oscillating circuit. Applying a steady DC voltage (e.g., pulling a GPIO pin HIGH) produces a single, fixed-pitch tone (typically ~2 kHz). Simple to code, but limited to basic beeps or chirps.
- Passive Buzzer: Lacks an internal oscillator and requires an AC or Pulse Width Modulation (PWM) signal to produce sound. By altering the input frequency, it can generate dynamic pitches, tones, and musical melodies.

Sensor-Triggered Alarm

```
int buzzerPin = 25;
int ldrPin = 34;
int threshold = 1000;

void setup() {
  pinMode(buzzerPin, OUTPUT);
}
```

```
void loop() {  
  int lightValue = analogRead(ldrPin);  
  if (lightValue < threshold) {  
    digitalWrite(buzzerPin, HIGH); // sound alarm when dark  
  } else {  
    digitalWrite(buzzerPin, LOW);  
  }  
}
```

Calibrate the threshold value based on earlier LDR observations, creating a simple "dark alarm" system.

POST-TEST:

Direction: Read each question carefully and select the letter corresponding to the correct answer.

1. What process occurs inside protocol-based digital sensors like the DHT11 before sending data to the ESP32?
 - A) Internal ADC chip converts and calibrates raw analog readings
 - B) Direct output of uncalibrated analog resistance values
 - C) Continuous voltage steps from 0V to 5V without protocol conversion
 - D) Relay contacts physically open to transmit data
2. Which library dependency must be installed alongside the main DHT sensor library in the Arduino IDE?
 - A) Adafruit SSD1306
 - B) Adafruit GFX Library
 - C) Wire.h
 - D) Adafruit Unified Sensor
3. In the declaration `Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);`, what does -1 specify?
 - A) The I2C address parameter
 - B) That no hardware reset pin is assigned
 - C) The default baud rate setting
 - D) The contrast level of the OLED screen
4. Why is threshold calibration required when programming a dark-alarm using an LDR and a buzzer?
 - A) To map the I2C bus address (e.g., 0x3C)
 - B) To set the exact analog voltage boundary for triggering the alarm
 - C) To initialize the single-wire communication timings
 - D) To step up the operating voltage from 3.3V to 5V
5. Which pins on an ESP32 board handle default I2C communication for modules like the SSD1306 OLED display?
 - A) GPIO 15 and GPIO 27
 - B) Pin 34 and Pin 25
 - C) SDA and SCL pins
 - D) VCC and GND pins
6. What is the primary safety risk when wiring external loads to a mechanical relay module?

- A) Reverse polarity on the LDR sensor
 - B) Direct contact with high-voltage AC mains on load terminals
 - C) Serial Monitor protocol mismatch
 - D) Exceeding the I2C clock speed
7. What happens if a 5V logic signal is directly connected to an unshielded 3.3V ESP32 GPIO pin?
- A) The GPIO pin converts it to an analog signal
 - B) It may permanently damage the microcontroller pin
 - C) The baud rate automatically doubles
 - D) The sensor switches from digital to analog mode
8. Which line of code initializes I2C serial communications for the OLED screen?
- A) `Wire.begin() / display.begin()`
 - B) `pinMode(OLED, INPUT)`
 - C) `analogRead(SSD1306)`
 - D) `digitalWrite(0x3C, HIGH)`
9. Why is a voltage divider circuit necessary when using a basic LDR with an analog pin?
- A) To convert resistance changes into variable voltage levels
 - B) To convert digital signals into I2C packets
 - C) To generate a PWM frequency for the buzzer
 - D) To prevent the PIR sensor from triggering false alarms
10. What will `digitalRead()` return when connected to a PIR sensor that detects movement?
- A) An analog value between 0 and 4095
 - B) Logic HIGH (1)
 - C) Logic LOW (0)
 - D) A string reading "Motion"

Pre-Test Answers:

1. B — `analogRead()` reads continuous voltage converted by the ADC.
2. C — I2C relies on SDA and SCL lines.
3. B — LDR resistance decreases as light level increases.
4. A — Relays act as switches for high-power devices using low-power control signals.
5. C — PIR sensors output binary logic levels (HIGH when motion detected).
6. B — ESP32 operates natively on 3.3 V logic.
7. A — `pinMode()` configures pin modes.
8. B — Buzzers convert electrical energy into sound.
9. B — DHT11 uses a single-wire custom pulse train.
10. B — Libraries provide functions to handle complex sensor protocols.

Post-Test Answers:

1. A — Internal ADCs digitize and calibrate measurements inside protocol-based sensors.
2. D — Adafruit Unified Sensor is the required dependency.
3. B — -1 indicates that no external reset pin is connected to the display.
4. B — Thresholds set the specific analog point where low/high light triggers the actuator.
5. C — I2C modules communicate via dedicated SDA (Data) and SCL (Clock) pins.
6. B — Exposure to high-voltage AC mains poses serious shock hazards.
7. B — Overvoltage on 3.3V logic pins can permanently damage the ESP32.
8. A — `display.begin()` initializes the screen and wire interface.
9. A — A voltage divider converts resistance variations into voltage changes measurable by ADC.
10. B — Active motion triggers a logic HIGH output on PIR sensors.

REFERENCES

- Adafruit Industries. (2023). *DHT Series Sensor Library Documentation*. GitHub.
- Espressif Systems. (2024). *ESP32 Series Datasheet & Hardware Reference*. Espressif Systems.
- Monk, S. (2017). *Programming Arduino: Getting Started with Sketches* (2nd ed.). McGraw-Hill Education.
- Schwartz, M. (2021). *Internet of Things with ESP32*. Packt Publishing.